Höhere technische Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs Bundesschulzentrum

LINUX Shell

Grundlagen der BASH

Erstellt von:

Michel Rögl-Brunner, BEd

13. April 2026

Inhaltsverzeichnis

1 Grundlagen **2**

1.1 Was ist Linux? . . . 2

1.2 Aufbau des Systems . . . 2

1.2.1 Kernel, Treiber und Systemaufrufe . . . 2

1.2.2 Shell und Prozesse . . . 3

1.3 Benutzer, Gruppen und Rechte . . . 3

1.4 Dateien und Dateirechte . . . 3

1.5 Dateisystem-Grundlagen . . . 4

1.6 Verzeichnisstruktur unter Linux . . . 4

1.7 Distributionen und Desktop-Umgebungen . . . 5

2 Die Linux-Shell (Bash) **6**

2.1 Grundidee der Shell . . . 6

2.2 Navigation und Orientierung . . . 6

2.3 Dateien lesen, erstellen und verwalten . . . 7

2.4 Variablen und Ausgabe . . . 7

2.5 Umleitungen und Pipes . . . 7

2.6 Filtern, Hilfe und Administration . . . 8

3 Debian mit WSL unter Windows einrichten **9**

3.1 Voraussetzung . . . 9

3.2 Installation von WSL mit Debian . . . 9

3.3 Erster Start und Benutzer . . . 9

3.4 Kurzer Funktionstest . . . 10

4 Aufgaben zur Linux-Shell **11**

5 Linux Shell Skripte **12**

5.1 Einführung in Bash-Skripte . . . 12

5.2 Parameter und Exit-Codes . . . 12

5.3 Bedingungen mit test und if . . . 13

5.4 Schleifen in Bash . . . 13

5.5 Funktionen und Variablenbereiche . . . 13

5.6 Debugging und praktische Hinweise . . . 14

6 Aufgaben zu Shell-Skripten **15**
6.1 Leicht 15
6.2 Mittel 15
6.3 Schwer 16

7 Beispieldateien und Download **17**

8 Abgabe **18**

1 Grundlagen

1.1 Was ist Linux?

Linux ist ein freies, plattformunabhängiges Mehrbenutzer-Betriebssystem. In der Praxis verwendet man fast immer eine Linux-Distribution. Eine Distribution kombiniert den Linux-Kernel mit Programmen, Paketverwaltung und Werkzeugen zu einem sofort nutzbaren Gesamtsystem.

Linux trennt klar zwischen Hardware und Anwendungen. Programme greifen nicht direkt auf Geräte zu, sondern nutzen Systemaufrufe. Der Kernel nimmt diese Aufrufe entgegen, verwaltet Ressourcen und steürt über Treiber die Kommunikation mit der Hardware.

1.2 Aufbau des Systems

1.2.1 Kernel, Treiber und Systemaufrufe

Der Kernel ist das Kernstück des Betriebssystems. Er stellt Funktionen für Speicherverwaltung, Prozessverwaltung, Dateisysteme und Gerätezugriff bereit. Wenn ein Programm z. B. Text ausgeben will, startet es einen Systemaufruf. Erst das Betriebssystem führt die Aktion auf der Hardware aus.

Treiber sind die Schnittstelle zu konkreten Geräten. Dadurch bleibt Software weitgehend unabhängig von einzelner Hardware, weil der Zugriff immer über den Kernel und die passenden Treiber erfolgt.

1.2.2 Shell und Prozesse

Die Shell ist eine textbasierte Benutzerschnittstelle. In dieser Arbeit wird die **BASH** (Bourne Again Shell) verwendet, die auf vielen Linux-Systemen Standardshell ist. Befehle in der Shell werden vom System verarbeitet und als entsprechende Operationen ausgeführt.

Linux ist ein Multitasking-System: Mehrere Prozesse können scheinbar gleichzeitig laufen. Ein Scheduler teilt die CPU-Zeit in sehr kleine Zeitscheiben auf und weist sie Prozessen zu. Dadurch bleibt das System reaktionsfähig, auch wenn einzelne Programme viel Rechenzeit benötigen.

1.3 Benutzer, Gruppen und Rechte

Linux ist ein Multiuser-System. Jeder Benutzer besitzt eine eindeutige **User ID (UID)** und ist mindestens einer Gruppe mit **Group ID (GID)** zugeordnet. Über Benutzer- und Gruppenzugehörigkeit wird gesteürt, wer welche Dateien lesen, verändern oder ausführen darf.

Eine Sonderrolle hat der Benutzer **root** mit UID `0`. Root besitzt nahezu uneingeschränkten Zugriff auf das gesamte System. Deshalb arbeitet man im Alltag möglichst mit einem normalen Benutzerkonto und verwendet administrative Rechte nur gezielt.

1.4 Dateien und Dateirechte

Datei- und Verzeichnisnamen sind unter Linux case-sensitive: `DATEI`, `datei` und `Datei` sind verschiedene Namen. Punkte sind normale Zeichen im Dateinamen. Namen, die mit einem Punkt beginnen, gelten als versteckt.

Neben regulären Dateien gibt es weitere Dateiarten, unter anderem Verzeichnisse, symbolische Links und Gerätedateien. Unter Linux gilt grundsätzlich: Vieles wird als Datei modelliert.

Für Zugriffsrechte wird häufig die Kurzschreibweise `rwx` verwendet:

- `r` (read): lesen
- `w` (write): schreiben
- `x` (execute): ausführen

Diese Rechte werden getrennt für Eigentümer, Gruppe und andere Benutzer vergeben.

1.5 Dateisystem-Grundlagen

Ein Dateisystem organisiert Daten auf einem Datenträger in Blöcken (bzw. Clustern) und verwaltet, wo Dateien physisch liegen. Die Wahl des Dateisystems beeinflusst unter anderem Performance, Zuverlässigkeit und Wiederherstellbarkeit nach Fehlern.

Unter Linux sind ext-Dateisysteme weit verbreitet, insbesondere ext4. Journalfähige Dateisysteme protokollieren Metadaten-Änderungen und erhöhen so die Konsistenz nach Abstürzen.

1.6 Verzeichnisstruktur unter Linux

Im Gegensatz zu Windows mit mehreren Laufwerksbuchstaben besitzt Linux einen einzigen Verzeichnisbaum ab dem Wurzelverzeichnis `/`. Weitere Datenträger werden an passenden Stellen in diesen Baum eingehangen (gemountet).

Wichtige Verzeichnisse sind zum Beispiel:

- `/bin`: grundlegende Programme
- `/boot`: Kernel und Startdateien
- `/dev`: Gerätedateien
- `/etc`: systemweite Konfiguration
- `/home`: Home-Verzeichnisse normaler Benutzer
- `/root`: Home-Verzeichnis des Benutzers root
- `/tmp`: temporäre Dateien
- `/usr`: Programme und systemweite Ressourcen
- `/var`: veränderliche Daten, z. B. Logs

Diese einheitliche Struktur ist ein grosser Vorteil von Linux/Unix-Systemen, weil man sich auf unterschiedlichen Distributionen schnell zurechtfindet.

1.7 Distributionen und Desktop-Umgebungen

Linux liegt nicht als ein einziges fertiges Produkt vor, sondern in vielen Distributionen. Bekannte Beispiele sind Debian, Fedora, openSUSE, Ubuntu und Linux Mint. Sie unterscheiden sich unter anderem bei Paketqüllen, Release-Modell, Vorkonfiguration und Zielgruppe.

Auch die grafische Oberfläche ist frei wählbar. Verbreitete Desktop-Umgebungen sind GNOME, KDE, XFCE, MATE und Cinnamon. Für den Shell-Unterricht ist jedoch vor allem relevant, dass alle diese Systeme eine leistungsfähige Kommandozeile mit BASH bereitstellen.

2 Die Linux-Shell (Bash)

2.1 Grundidee der Shell

Die Shell ist die textbasierte Schnittstelle zwischen Benutzer und Betriebssystem. In dieser Arbeit wird die Bash (Bourne Again Shell) verwendet. Befehle werden in der Kommandozeile eingegeben und direkt ausgeführt. Viele Verwaltungs- und Automatisierungsaufgaben lassen sich damit schneller erledigen als über eine grafische Oberfläche.

Nach dem Start befindet man sich üblicherweise im Home-Verzeichnis. Mit `pwd` (print working directory) kann jederzeit geprüft werden, in welchem Verzeichnis man aktüll arbeitet.

2.2 Navigation und Orientierung

Die wichtigsten Befehle für den Einstieg sind:

- `ls`: listet Dateien und Verzeichnisse auf
- `ls -l`: detaillierte Ansicht mit Rechten, Besitzer und Grösse
- `ls -a`: zeigt auch versteckte Dateien (Namen mit Punkt am Anfang)
- `cd <verzeichnis>`: wechselt in ein Verzeichnis
- `cd ..`: wechselt eine Ebene nach oben
- `cd /`: wechselt ins Wurzelverzeichnis
- `cd`: wechselt ins Home-Verzeichnis

Zum Suchen von Dateien dient `find`, zum Beispiel `find /etc -name fonts.conf`. Damit kann man gezielt in Verzeichnisbäumen nach Dateinamen suchen.

2.3 Dateien lesen, erstellen und verwalten

Zum Lesen von Textdateien eignet sich `less`. Die Datei wird seitenweise angezeigt, was bei grösseren Inhalten deutlich übersichtlicher ist als eine direkte Komplettausgabe.

Dateien und Ordner werden mit folgenden Befehlen erstellt:

- `mkdir <name>`: neüs Verzeichnis
- `touch <name>`: leere Datei

Für die Verwaltung werden vor allem drei Befehle verwendet:

- `cp <qülle> <ziel>`: kopieren
- `mv <qülle> <ziel>`: verschieben oder umbenennen
- `rm <datei>` und `rmdir <verzeichnis>`: löschen

2.4 Variablen und Ausgabe

Mit `echo` werden Texte oder Variableninhalte ausgegeben. Variablen lassen sich direkt in der Shell setzen:

- `MeinText=Ïch bin wiederverwendbar"`
- `echo $MeinText`

Wichtige Umgebungsvariablen sind zum Beispiel `$HOME`, `$HOSTNAME`, `$LOGNAME` und `$UID`. Sie stellen Systeminformationen bereit und sind besonders in Shell-Skripten nützlich.

2.5 Umleitungen und Pipes

Die Standardausgabe kann in Dateien umgeleitet werden:

- `>`: schreibt neu und überschreibt vorhandenen Inhalt
- »: hängt an vorhandenen Inhalt an

Beispiel: `ls -l > info.txt`

Fehlerausgaben lassen sich getrennt behandeln, z. B. mit `2> /dev/null`. Mit `|` (Pipe) wird die Ausgabe eines Befehls als Eingabe für den nächsten genutzt, z. B. `ls -l | less`.

2.6 Filtern, Hilfe und Administration

`cat` gibt Dateiinhalt auf der Standardausgabe aus. In Kombination mit Wildcards oder Umleitung können auch mehrere Dateien zusammengeführt werden.

`grep` filtert Zeilen nach Mustern. Das ist hilfreich, um in grossen Ausgaben schnell relevante Zeilen zu finden oder irrelevante auszuschliessen.

Für Dokumentation und Hilfe stehen drei Werkzeuge bereit:

- `man <befehl>`: Handbuchseite
- `info <befehl>`: erweiterte Info-Dokumentation
- `help`: integrierte Hilfe der Shell

Administrationsaufgaben werden mit erhöhten Rechten ausgeführt. Dazu dienen je nach System `su` oder `sudo`. Im Alltag sollte weiterhin mit normalen Benutzerrechten gearbeitet werden.

3 Debian mit WSL unter Windows einrichten

3.1 Voraussetzung

Diese Anleitung gilt für einen Windows-PC mit PowerShell.

3.2 Installation von WSL mit Debian

1. Öffne **PowerShell als Administrator**.
2. Führe folgenden Befehl aus:
   - `wsl -install -d debian`
3. Starte den PC neu, falls Windows dazu auffordert.
4. Starte danach **Debian** mit `wsl` in PowerShell.

3.3 Erster Start und Benutzer

Bei der installation richtet sich Debian ein und fragt nach einem Linux-Benutzer.

- Wähle als Benutzername bitte deinen **Vornamen**.
- Vergib ein Passwort und merke es dir gut.

Wichtig bei der Passworteingabe in Linux:

- Es werden **keine Zeichen** angezeigt.
- Es erscheinen auch **keine Sternchen**.
- Die Eingabe funktioniert trotzdem normal. Nach dem Tippen einfach mit ENTER bestätigen.

3.4 Kurzer Funktionstest

Nach erfolgreicher Einrichtung kannst du in Debian zum Test folgende Befehle ausführen:

- `whoami`
- `pwd`
- `ls`

Wenn diese Befehle funktionieren, ist dein Debian-System unter WSL betriebsbereit.

Vergiss nicht in dein Home-Verzeichniss zu wechslen. `cd`

4 Aufgaben zur Linux-Shell

1. Lege in deinem Home-Verzeichnis ein Verzeichnis mit dem Namen `COPR` an.
2. Gib den String `Hello, world!` in der Shell aus.
3. Prüfe mit `man`, wie `echo` offiziell beschrieben wird.
4. Gib die Strings `fee`, `fie`, `fö` und `fum` aus und nutze dabei die Befehls-Historie mit den Pfeiltasten.
5. Lösche die sichtbare Ausgabe in der Kommandozeile.
6. Gib den Text `Use man echo". Tippe: Doppelte Anführungszeichen verwenden.` korrekt aus.
7. Finde mit `help` heraus, wie das Terminal für 5 Sekunden pausiert werden kann (`sleep`).
8. Erzeuge mit `echo` und Umleitung zwei Dateien `line_1.txt` und `line_2.txt` mit beliebigem Inhalt.
9. Kombiniere beide Dateien mit `cat` in eine dritte Datei.
10. Zeige den Inhalt der dritten Datei mit `less` an.
11. Liste alle nicht versteckten Dateien auf, die mit `l` beginnen.
12. Liste alle Dateien inklusive versteckter Dateien in der langen Form auf.
13. Erstelle `foo.txt` mit dem Inhalt `hello world`, kopiere die Datei und nenne die Kopie `bar.txt`.
14. Notiere die nötigen Befehle, um eine Datei `foo` in `bar` umzubenennen und danach eine Kopie `foobar` zu erzeugen.

5 Linux Shell Skripte

5.1 Einführung in Bash-Skripte

Ein Bash-Skript ist eine Textdatei mit Shell-Befehlen, die nacheinander ausgeführt werden. Am Beginn steht üblicherweise der sogenannte Shebang:

- `#!/bin/bash`

Damit wird festgelegt, mit welcher Shell das Skript interpretiert werden soll. Danach können beliebige Befehle folgen, z. B. `echo 'Servus Linux'`.

Damit ein Skript direkt gestartet werden kann, muss es ausführbar sein, zum Beispiel mit `chmod +x <datei>`. Der Start erfolgt dann über:

- `./<dateiname>`
- `./<dateiname> <parameter1> <parameter2>`

5.2 Parameter und Exit-Codes

Beim Aufruf können Parameter übergeben werden. Im Skript werden sie über Stellungsparameter angesprochen:

- `$0`: Name des Skripts
- `$1`, `$2`, ...: übergebene Parameter
- `$#`: Anzahl aller Parameter
- `$?`: Exit-Code des zuletzt ausgeführten Befehls

Ein Exit-Code von `0` bedeutet meist Erfolg, ein Wert ungleich `0` weist auf einen Fehler hin. Mit `exit <code>` kann ein Skript einen eigenen Rückgabewert setzen.

5.3 Bedingungen mit test und if

Für logische Prüfungen werden in Bash meist `test` oder die Kurzschreibweise `[ ... ]` verwendet. Typische Prüfungen sind:

- String-Vergleich: `==`
- Zahlenvergleich: `-eq`, `-lt`, `-gt`
- Datei existiert: `-e`
- Verzeichnis existiert: `-d`
- String leer/nicht leer: `-z`, `-n`

Darauf baut die `if`-Anweisung auf:

- `if [ <bedingung> ]; then ... else ... fi`

Wichtig ist die korrekte Schreibweise mit Leerzeichen innerhalb der eckigen Klammern.

5.4 Schleifen in Bash

Für Wiederholungen werden vor allem `for`- und `while`-Schleifen verwendet.

for eignet sich gut für bekannte Wiederholungszahlen oder Listen, zum Beispiel zum Durchlaufen von Parametern. **while** wird genutzt, solange eine Bedingung wahr ist, etwa für Zähler-Schleifen.

Beide Schleifentypen sind zentrale Werkzeuge, um wiederkehrende Aufgaben in Skripten kompakt und sauber zu lösen.

5.5 Funktionen und Variablenbereiche

Funktionen strukturieren Skripte und machen Code wiederverwendbar. Parameter werden innerhalb von Funktionen ebenfalls über `$1`, `$2`, ... angesprochen.

Rückgaben erfolgen mit `return`. Für den Variablenbereich gilt:

- globale Variablen sind im ganzen Skript sichtbar
- lokale Variablen werden in Funktionen mit `local` deklariert

So lassen sich Seiteneffekte vermeiden und Skripte besser wartbar halten.

5.6 Debugging und praktische Hinweise

Zum Nachvollziehen der Befehlsausführung ist `bash -x <skript>` sehr hilfreich. Damit wird beim Lauf angezeigt, welche Zeilen gerade ausgeführt werden und welche Werte Variablen annehmen.

Für robustere Skripte sollten Eingaben früh geprüft werden (Anzahl und Gültigkeit der Parameter, Existenz von Dateien/Verzeichnissen, Rechteprüfung bei Admin-Aufgaben).

6 Aufgaben zu Shell-Skripten

6.1 Leicht

1. **Einfache Rechenmaschine (`rechner.sh`)**
   Nimm zwei Zahlen als Parameter entgegen und berechne Summe, Differenz, Produkt und Division in einer gut lesbaren Ausgabe.
   Beispiel Eingabe: `./rechner.sh 10 5`
   Beispiel Ausgabe: `Summe: 15, Differenz: 5, Produkt: 50, Division: 2`
   Erweiterung: Gib eine Fehlermeldung aus, wenn Parameter fehlen oder ungültig sind.

2. **Dateizähler in einem Verzeichnis (`count_files.sh`)**
   Nimm ein Verzeichnis als Parameter, prüfe dessen Existenz und zähle Dateien sowie Unterordner.
   Beispiel Eingabe: `./count_files.sh ./testdaten`
   Beispiel Ausgabe: `Dateien: 8, Ordner: 3`
   Erweiterung: Zähle optional nur Dateien mit einer bestimmten Endung (z. B. `.txt`).

6.2 Mittel

3. **Backup-Skript (`backup.sh`)**
   Kopiere ein angegebenes Verzeichnis in ein Backup-Verzeichnis und versehe den Zielordner mit Zeitstempel (z. B. `backup_2026-02-27_14-30`).
   Beispiel Eingabe: `./backup.sh ./projekt ./backups`
   Beispiel Ausgabe: `Backup erstellt: ./backups/backup_2026-02-27_14-30`
   Erweiterung: Lösche alte Backups automatisch (älter als 7 Tage).

4. **Logfile-Analyzer (`log_analyzer.sh`)**
   Analysiere eine Logdatei und zähle, wie oft `ERROR`, `WARNING` und `INFO` vorkommen. Gib die Ergebnisse formatiert aus.
   Beispiel Eingabe: `./log_analyzer.sh app.log`
   Beispiel Ausgabe: `ERROR: 4, WARNING: 7, INFO: 21`
   Erweiterung: Werte optional nur einen per Parameter gewählten Log-Level aus.
   Input-Datei: `BEISPIELDATEIN/LINUX/Aufgabe-Logfile-Analyzer/app.log`
   Direktlink: `https://raw.githubusercontent.com/michelroegl-brunner/HTL-Braunau/ma`

5. **Shell-Skript zum Bewegen von Dateien (`mvprot.sh`)**
   Entwickle ein Skript, das im aktüllen Verzeichnis alle Dateien einer angegebenen Endung in ein neu erstelltes Unterverzeichnis verschiebt. Das Skript besitzt drei Parameter: Dateiendung, Name des Unterverzeichnisses und ein Flag, ob ein Protokoll erstellt werden soll. Bei aktivem Protokoll sollen Anzahl und Namen der verschobenen Dateien in eine Protokolldatei im Qüllverzeichnis geschrieben werden.
   Beispiel Eingabe: `./mvprot.sh txt archiv ja`
   Beispiel Ausgabe: `5 Dateien nach ./archiv verschoben. Protokoll: ./mvprot.log`
   Hinweis: Die Anzahl kann z. B. über Kommandosubstitution ermittelt werden, etwa mit `AnzahlDateien=$(ls *.txt | wc -w)`.

6.3 Schwer

6. **Benutzer-Management-Skript (`user_manager.sh`)**
   Prüfe, ob das Skript mit Root-Rechten läuft, und unterstütze das Hinzufügen sowie Löschen von Benutzern (`add`/`delete`). Prüfe dabei auch, ob ein Benutzer bereits existiert.
   Beispiel Eingabe: `sudo ./user_manager.sh add max`
   Beispiel Ausgabe: `Benutzer max wurde erfolgreich angelegt.`
   Erweiterung: Passwort automatisch generieren und Benutzer einer Gruppe zuordnen.

7 Beispieldateien und Download

Wenn fuer eine Aufgabe Beispieldateien benoetigt werden, liegen diese im Repository unter:

- `BEISPIELDATEIN/LINUX/<AUFGABE>/...`
- `BEISPIELDATEIN/POWERSHELL/<AUFGABE>/...`

Beispiele fuer Linux-Aufgaben:

- `BEISPIELDATEIN/LINUX/Aufgabe-8/line_1.txt`
- `BEISPIELDATEIN/LINUX/Aufgabe-8/line_2.txt`
- `BEISPIELDATEIN/LINUX/Aufgabe-13/foo.txt`
- `BEISPIELDATEIN/LINUX/Aufgabe-Logfile-Analyzer/app.log`

Download direkt ueber GitHub:

- Einzeldatei: Datei oeffnen, `Raw` klicken, danach speichern.
- Ganze Sammlung: `https://github.com/michelroegl-brunner/HTL-Braunau/archive/ref`
- Raw-Schema: `https://raw.githubusercontent.com/michelroegl-brunner/HTL-Braunau`

8 Abgabe

Die Ausarbeitung ist als **PDF-Datei** abzugeben.

Pro Aufgabe ist ein Screenshot der Ausgabe des Befehls zu machen. Neu verwendete Befehle sind in kurzen Stichworten zu beschreiben.

Bitte lade deine fertige PDF in die entsprechende **Teams-Aufgabe** hoch.

Test